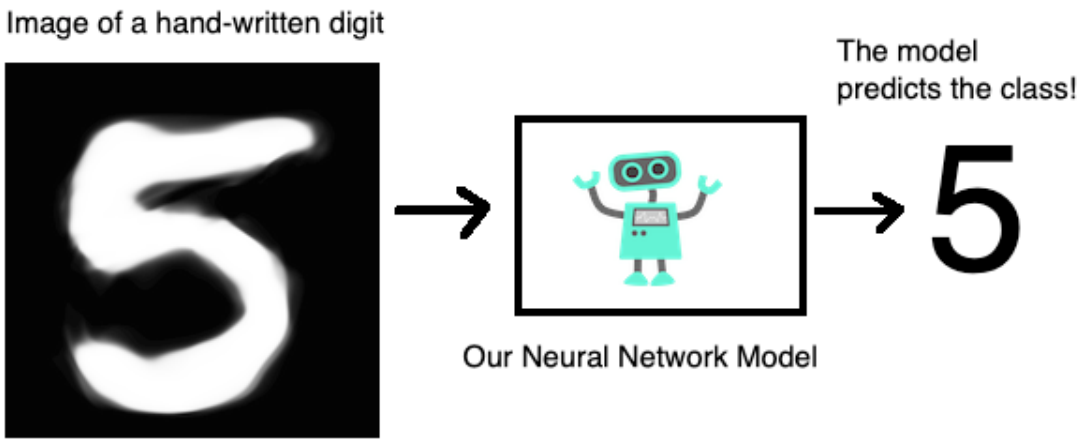
Image Classification with Tensorflow

June 17, 2026

1 Task 1: Introduction

Welcome to Basic Image Classification with TensorFlow.

This graph describes the problem that we are trying to solve visually. We want to create and train a model that takes an image of a hand written digit as input and predicts the class of that digit, that is, it predicts the digit or it predicts the class of the input image.



1.0.1 Import TensorFlow

[6]:
```
import tensorflow as tf

#tf.logging.set_verbosity(tf.logging.ERROR)
print('Using TensorFlow version', tf.__version__)
```

```
Using TensorFlow version 2.10.0
```

2 Task 2: The Dataset

2.0.1 Import MNIST

[7]:
```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434 [==============================] - 0s 0us/step
```

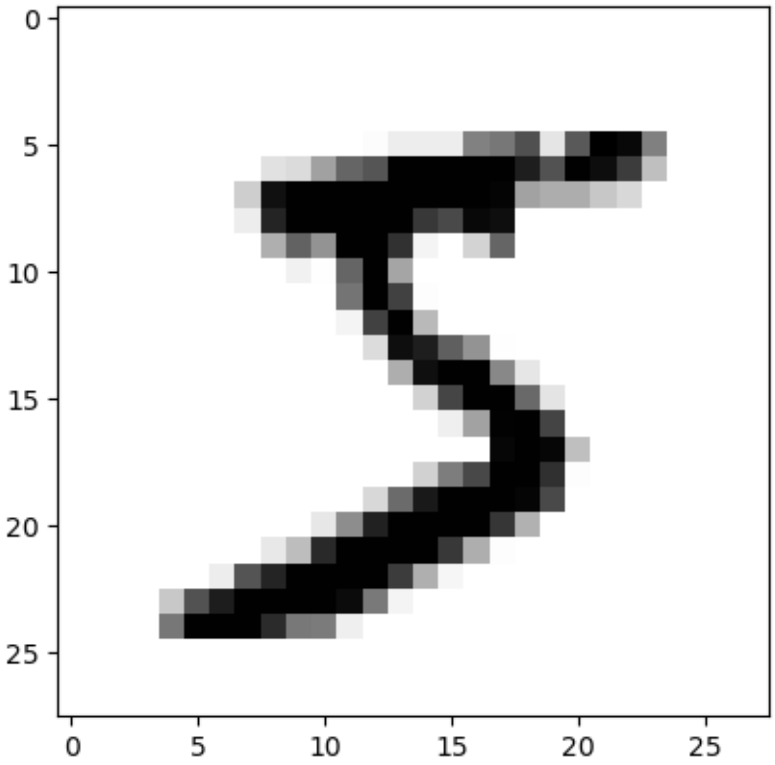
2.0.2 Shapes of Imported Arrays

[8]:
```
print('x_train shape:', x_train.shape)
print('y_train shape:', y_train.shape)
print('x_test shape:', x_test.shape)
print('y_test shape:', y_test.shape)
#(no.of example, no.of rows, no. of columns)
```

```
x_train shape: (60000, 28, 28)
y_train shape: (60000,)
x_test shape: (10000, 28, 28)
y_test shape: (10000,)
```

2.0.3 Plot an Image Example

[9]:
```
from matplotlib import pyplot as plt
%matplotlib inline

plt.imshow(x_train[0], cmap='binary')
plt.show()
```



2.0.4 Display Labels

[10]:
```
y_train[0]
```

[10]: 5

[11]:
```
# prints the classes that we have
print(set(y_train))
```

```
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

3 Task 3: One Hot Encoding

After this encoding, every label will be converted to a list with 10 elements and the element at index to the corresponding class will be set to 1, rest will be set to 0:

original label	one-hot encoded label
5	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
7	[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
1	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]

3.0.1 Encoding Labels

[14]:
```
from tensorflow.keras.utils import to_categorical

y_train_encoded = to_categorical(y_train)
y_test_encoded = to_categorical(y_test)
```

3.0.2 Validated Shapes

[15]:
```
#to check if the encoding worked we check the
#shape of the encoded tables
print('y_train_encoded shape:', y_train_encoded.shape)
print('y_test_encoded shape:', y_test_encoded.shape)
#this shows that each example is a 10 dimensional vector
```

```
y_train_encoded shape: (60000, 10)
y_test_encoded shape: (10000, 10)
```

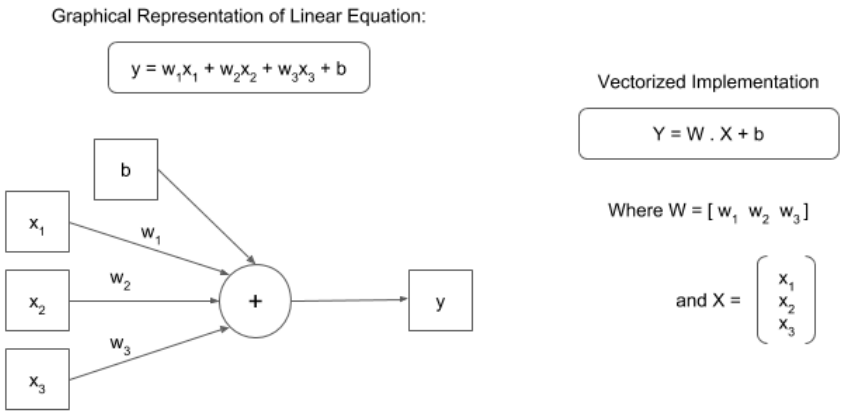
3.0.3 Display Encoded Labels

[18]:
```
y_train_encoded[0]
```

[18]:
```
array([0., 0., 0., 0., 0., 1., 0., 0., 0., 0.], dtype=float32)
```

4 Task 4: Neural Networks

4.0.1 Linear Equations



The above graph simply represents the equation:

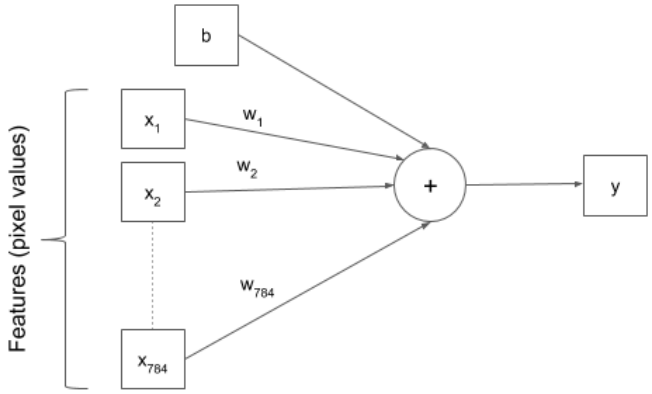
$$y = w1 * x1 + w2 * x2 + w3 * x3 + b \tag{1}$$

Where the `w1, w2, w3` are called the weights and `b` is an intercept term called bias. The equation can also be *vectorised* like this:

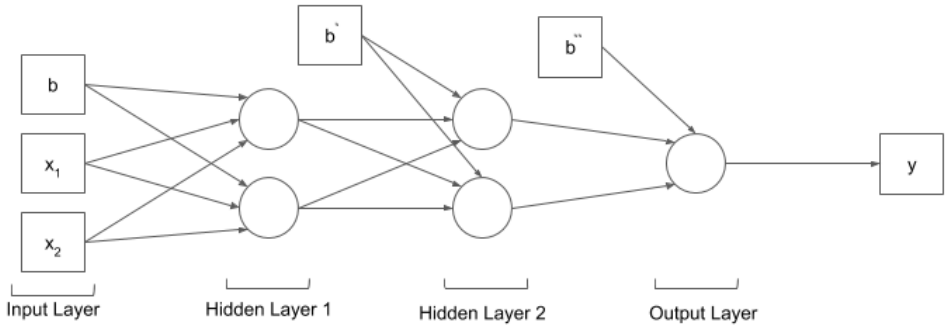
$$y = W.X + b \tag{2}$$

Where `X = [x1, x2, x3]` and `W = [w1, w2, w3].T`. The .T means *transpose*. This is because we want the dot product to give us the result we want i.e. `w1 * x1 + w2 * x2 + w3 * x3`. This gives us the vectorised version of our linear equation.

A simple, linear approach to solving hand-written image classification problem - could it work?



4.0.2 Neural Networks



This model is much more likely to solve the problem as it can learn more complex function mapping for the inputs and outputs in our dataset.

5 Task 5: Preprocessing the Examples

5.0.1 Unrolling N-dimensional Arrays to Vectors

[20]:
```
import numpy as np

x_train_reshaped = np.reshape(x_train, (60000,784))
x_test_reshaped = np.reshape(x_test, (10000,784))

print('x_train_reshaped shape:', x_train_reshaped.shape)
print('x_test_reshaped shape:', x_test_reshaped.shape)
#784 dimensional vector. Each value is a pixel in the image
```

```
x_train_reshaped shape: (60000, 784)
x_test_reshaped shape: (10000, 784)
```

5.0.2 Display Pixel Values

[22]:
```
print(set(x_train_reshaped[0]))
```

```
{0, 1, 2, 3, 9, 11, 14, 16, 18, 23, 24, 25, 26, 27, 30, 35, 36, 39, 43, 45, 46,
49, 55, 56, 64, 66, 70, 78, 80, 81, 82, 90, 93, 94, 107, 108, 114, 119, 126,
127, 130, 132, 133, 135, 136, 139, 148, 150, 154, 156, 160, 166, 170, 171, 172,
175, 182, 183, 186, 187, 190, 195, 198, 201, 205, 207, 212, 213, 219, 221, 225,
226, 229, 238, 240, 241, 242, 244, 247, 249, 250, 251, 252, 253, 255}
```

5.0.3 Data Normalization

[26]:
```
x_mean = np.mean(x_train_reshaped)
x_std = np.std(x_train_reshaped)

epsilon = 1e-10

x_train_norm = (x_train_reshaped - x_mean) / (x_std + epsilon)
x_test_norm = (x_test_reshaped - x_mean) / (x_std + epsilon)
```

5.0.4 Display Normalized Pixel Values

[27]:
```
print(set(x_train_norm[0]))
```

```
{-0.38589016215482896, 1.306921966983251, 1.17964285952926, 1.803310486053816,
1.6887592893452241, 2.8215433456857437, 2.719720059722551, 1.1923707702746593,
1.7396709323268205, 2.057868700961798, 2.3633385588513764, 2.096052433197995,
1.7651267538176187, 2.7960875241949457, 2.7451758812133495, 2.45243393406917,
```

```
0.02140298169794222, -0.22042732246464067, 1.2305545025108566,
0.2759611966059242, 2.210603629906587, 2.6560805059955555, 2.6051688630139593,
-0.4240738943910262, 0.4668798577869107, 0.1486820891519332, 0.3905123933145161,
1.0905474843114664, -0.09314821501064967, 1.4851127174188385,
2.7579037919587486, 1.5360243604004349, 0.07231462467953861,
-0.13133194724684696, 1.294194056237852, 0.03413089244334132,
1.3451056992194483, 2.274243183633583, -0.24588314395543887, 0.772349715676489,
0.75962180493109, 0.7214380726948927, 0.1995937321335296, -0.41134598364562713,
0.5687031437501034, 0.5941589652409017, 0.9378125553666773, 0.9505404661120763,
0.6068868759863008, 0.4159682148053143, -0.042236572029053274,
2.7706317027041476, 2.1342361654341926, 0.12322626766113501,
-0.08042030426525057, 0.16140999989733232, 1.8924058612716097,
1.2560103240016547, 2.185147808415789, 0.6196147867316999, 1.943317504253206,
-0.11860403650144787, -0.30952269768243434, 1.9942291472348024,
-0.2840668761916362, 2.6306246845047574, 2.286971094378982,
-0.19497150097384247, -0.39861807290022805, 0.2886891073513233,
1.7523988430722195, 2.3887943803421745, 2.681536327486354, 1.4596568959280403,
2.439706023323771, 2.7833596134495466, 2.490617666305367, -0.10587612575604877,
1.5614801818912332, 1.9051337720170087, 1.6123918248728295, 1.268738234747054,
1.9560454149986053, 2.6433525952501564, 1.026907930584471}
```

6 Task 6: Creating a Model

6.0.1 Creating the Model

[30]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(128, activation = 'relu', input_shape = (784,)),
    Dense(128, activation = 'relu'),
    Dense(10, activation = 'softmax')
])
```

6.0.2 Activation Functions

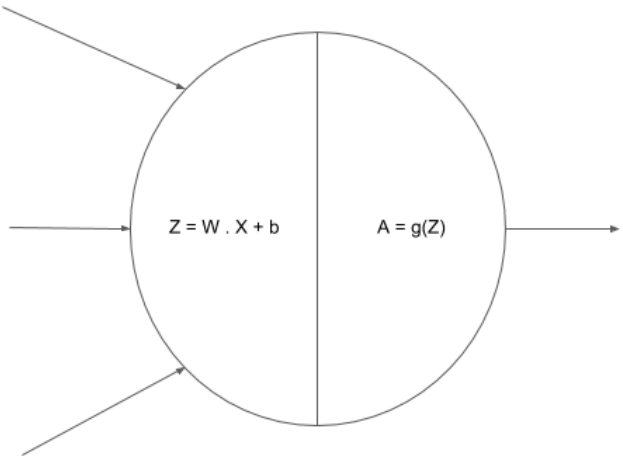
The first step in the node is the linear sum of the inputs:

$$Z = W.X + b \tag{3}$$

The second step in the node is the activation function output:

$$A = f(Z) \tag{4}$$

Graphical representation of a node where the two operations are performed:



6.0.3 Compiling the Model

[31]:
```
model.compile(
    optimizer = 'sgd',
    loss = 'categorical_crossentropy',
    metrics = ['accuracy']

)

model.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_6 (Dense)             (None, 128)               100480

 dense_7 (Dense)             (None, 128)               16512

 dense_8 (Dense)             (None, 10)                1290

=================================================================
Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0
_________________________________________________________________
```

7 Task 7: Training the Model

7.0.1 Training the Model

[32]:
```
model.fit(x_train_norm, y_train_encoded, epochs=3)
```

```
Epoch 1/3
1875/1875 [==============================] - 4s 2ms/step - loss: 0.3746 -
accuracy: 0.8908
Epoch 2/3
1875/1875 [==============================] - 3s 2ms/step - loss: 0.1859 -
accuracy: 0.9464
Epoch 3/3
1875/1875 [==============================] - 3s 2ms/step - loss: 0.1408 -
accuracy: 0.9592
```

[32]:
```
<keras.callbacks.History at 0x78457061e050>
```

7.0.2 Evaluating the Model

[35]:
```
loss, accuracy = model.evaluate(x_test_norm, y_test_encoded)
print('Test set accuracy: ', accuracy * 100)
```

```
313/313 [==============================] - 0s 1ms/step - loss: 0.1347 -
accuracy: 0.9576
Test set accuracy:  95.75999975204468
```

8 Task 8: Predictions

8.0.1 Predictions on Test Set

[36]:
```
preds = model.predict(x_test_norm)
print('Shape of preds: ', preds.shape)
```

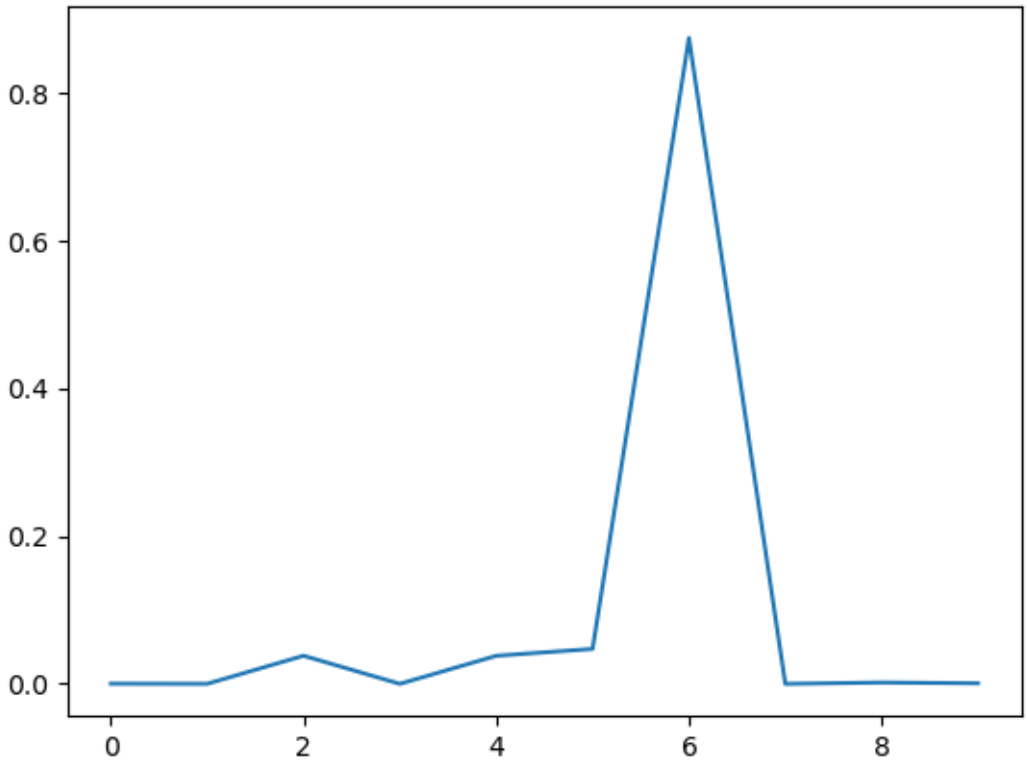
```
313/313 [==============================] - 0s 977us/step
Shape of preds:  (10000, 10)
```

8.0.2 Plotting the Results

[45]:
```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(12, 12))
start_index = 0

for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.grid(False)
```

```
    plt.xticks([])
    plt.yticks([])


    pred = np.argmax(preds[start_index + i])
    gt = y_test[start_index + i]

    # Green if correct, red if incorrect
    col = 'g' if pred == gt else 'r'

    plt.xlabel('i={}, pred={}, gt={}'.format(start_index + i, pred, gt),␣
↪color=col)
    plt.imshow(x_test[start_index + i], cmap='binary')


plt.show()
```

[43]:
```
plt.plot(preds[8])
#index 8 prediction is inaccurate
plt.show()
```


0.8
0.6
0.4
0.2
0.0
0
2
4
6
8